

Project #1: User Program

[CSE4070]

Instructors

Prof. Youngjae Kim (01)

Prof. Sungyong Park (02)

Teaching Assistants

Seong Jang (01), Taewan Noh (01)

Seungjae Lee (02), Myoungjoon Kim (02)

Fall 2026

Contents

1. Prerequisites

- Background
- How User Program Works
- Code Level Flow
- Virtual Memory
- System Calls

2. Requirements

- Process Termination Messages
- Argument Passing
- System Calls
- Base File System
- File System Call Implementation

3. Suggested Order of Implementation

4. Evaluation

5. Documentation

6. Submission

Prerequisites

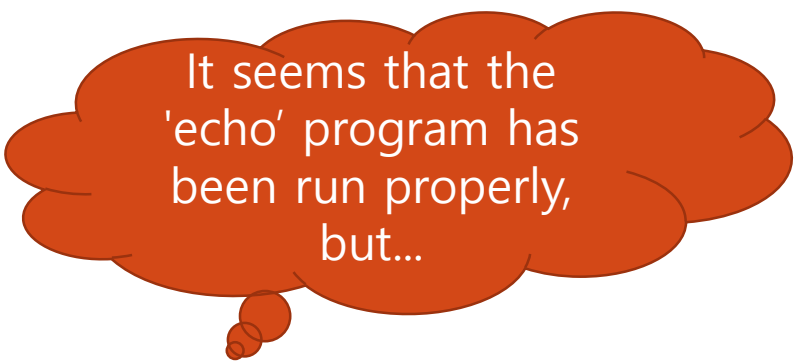
Background

- Pintos is a simple OS which can boot, execute an application, and power off.
- Run 'echo' application on Pintos first. (**Run 'make' in src/examples and src/userprog first**)

```
~/pintos/src/userprog $ pintos --filesystem-size=2 -p ../examples/echo -a echo -- -f -q run 'echo x'
```

Do not miss to type "--" (two hyphens!).

```
Formatting file system...done.
Boot complete.
Extracting ustar archive from scratch device into file system...
Putting 'echo' into the file system...
Erasing ustar archive...
Executing 'echo x':
Execution of 'echo x' complete.
Timer: 76 ticks
Thread: 0 idle ticks, 76 kernel ticks, 0 user ticks
hda2 (filesystem): 26 reads, 172 writes
hda3 (scratch): 83 reads, 2 writes
Console: 818 characters output
Keyboard: 0 keys pressed
Exception: 0 page faults
Powering off...
```



It seems that the
'echo' program has
been run properly,
but...

Background

- Pintos is a simple OS which can boot, execute an application, and power off.
- Run 'echo' application on Pintos first. (**Run 'make' in src/examples and src/userprog first**)

```
~/pintos/src/userprog $ pintos --filesystem-size=2 -p ../examples/echo -a echo -- -f -q run 'echo x'
```

Do not miss to type "--" (two hyphens!).

```
Formatting file system...done.
Boot complete.
Extracting ustar archive from scratch device into file system...
Putting 'echo' into the file system...
Erasing ustar archive...
Executing 'echo x':
Execution of 'echo x' complete.
Timer: 76 ticks
Thread: 0 idle ticks, 76 kernel ticks, 0 user ticks
hda2 (filesystem): 26 reads, 172 writes
hda3 (scratch): 83 reads, 2 writes
Console: 818 characters output
Keyboard: 0 keys pressed
Exception: 0 page faults
Powering off...
```

We can not see the result of 'echo x' because of **lack of implementation**.



The diagram shows a terminal window with a blue arrow pointing from the command 'echo x' to the output 'x'.

We should be able to see 'x'.

Background

- Why can't we see the result of 'echo' command?
- It is because, in current Pintos, **system call, system call handler, argument passing, and user stack** have not been implemented.
- Basically, current Pintos does not implement many OS functionalities, including those above.

Background

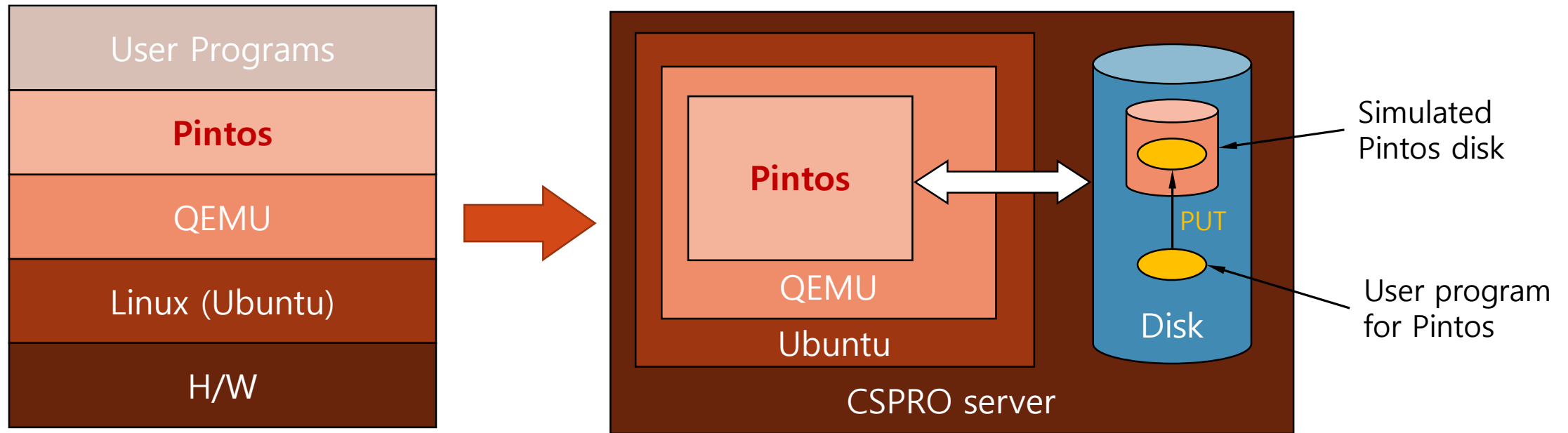
- In this project, students will have to make the Pintos **be able to execute user programs properly**.
- Students should work in the following directories and modify the following files:

	Files to be modified	Referenced files
src/ userprog	process.h / process.c syscall.h / syscall.c	pagedir.h / pagedir.c exception.h / exception.c
src/ threads	thread.h / thread.c	synch.h / synch.c vaddr.h
src/ devices	-	shutdown.h / shutdown.c input.h / input.c
src/ lib	syscall-nr.h user/syscall.h user/syscall.c	-

How User Program Works

Procedure:

1. Pintos can load and run regular ELF(Executable & Linkable Format) executables.
2. To run a user program, we must copy (put) the user program to the simulated file system disk.



How User Program Works

- Consider the previous example more in detail

```
~/pintos/src/userprog $ pintos --fileys-size=2 -p ../examples/echo -a echo -- -f -q run 'echo x'
```

"**--fileys-size=2**": Make simulated Pintos disk which consists of 2MB

"**-p ../examples/echo -a echo**": Copy '../examples/echo' into the simulated disk and **change the name** from '../examples/echo' to 'echo'

"**--**" **between echo and -f**: Separate pintos' options and kernel arguments

"**-f**": Pintos formats the simulated disk.

"**-q**": Pintos will be terminated after execution of 'echo'.

"**run 'echo x'**": Pintos will execute 'echo' with argument 'x'.

How User Program Works

- Consider the previous example more in detail

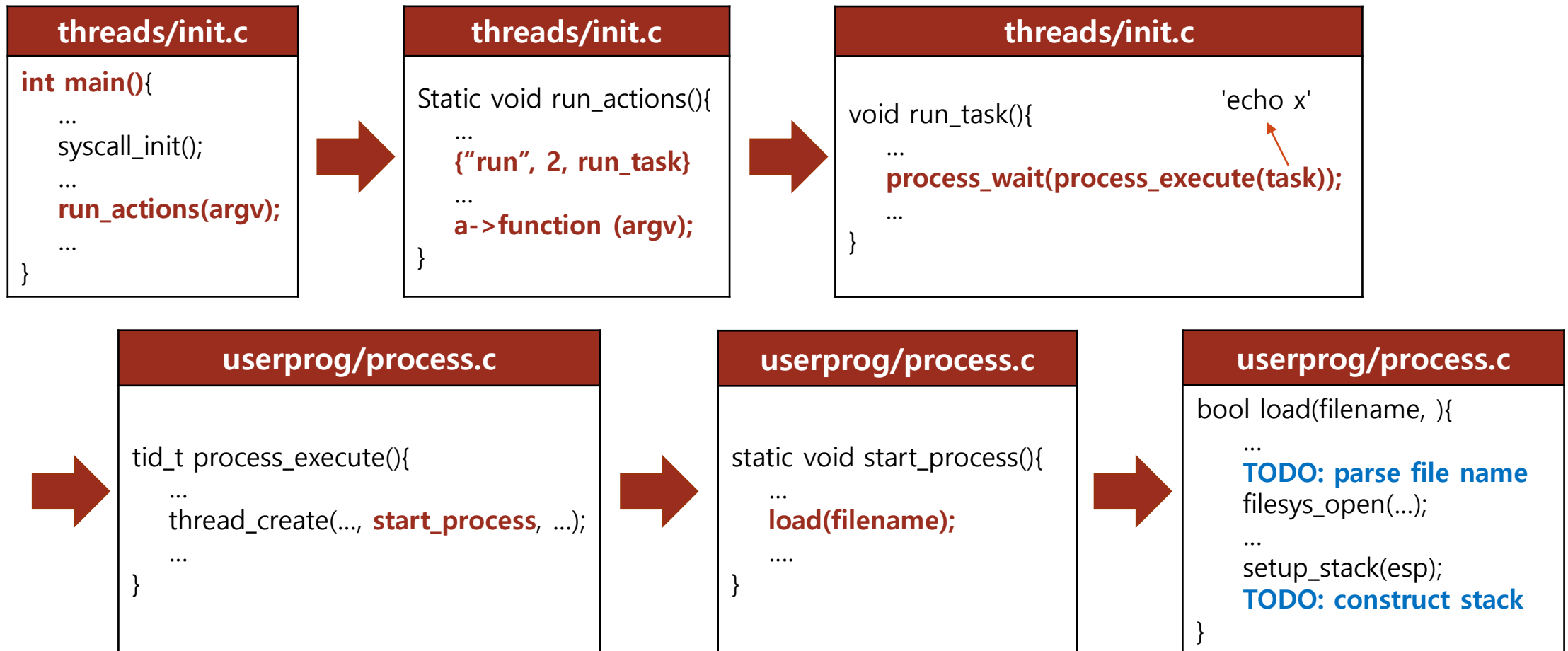
```
~/pintos/src/userprog $ pintos --fileys-size=2 -p ../examples/echo -a echo -- -f -q run 'echo x'
```

- ✓ 'echo' is the application that writes arguments to the **standard output**.
- ✓ Thus, 'echo' needs the I/O functionality provided by system call in the kernel.
- ✓ And, it also needs user stack implementation which stores arguments and passes them to kernel.
- ✓ But, Pintos has no implementation for **system calls** and **user stacks**.

That's why we were not able to see the result of 'echo x'.

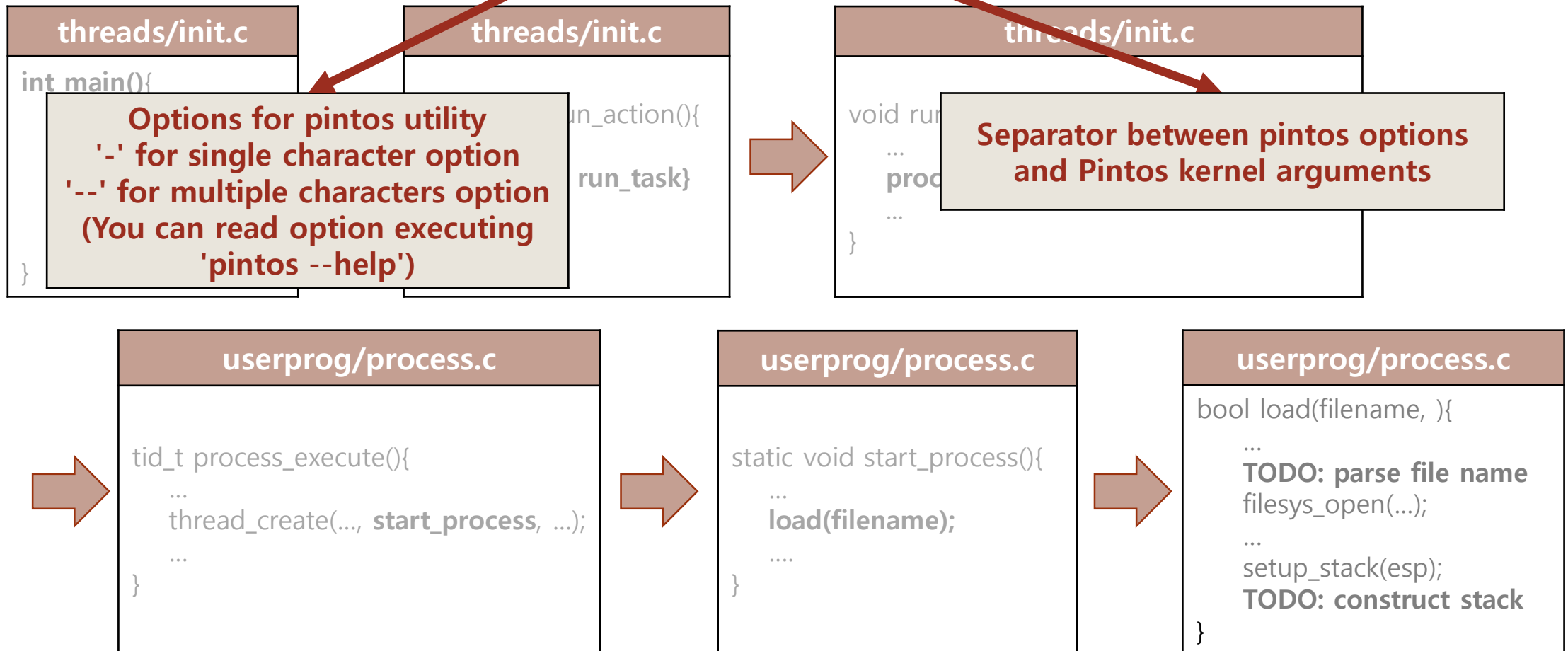
Code Level Flow

\$ pintos --filesystem-size=2 -p ../examples/echo -a echo -- -f -q run **'echo x'**



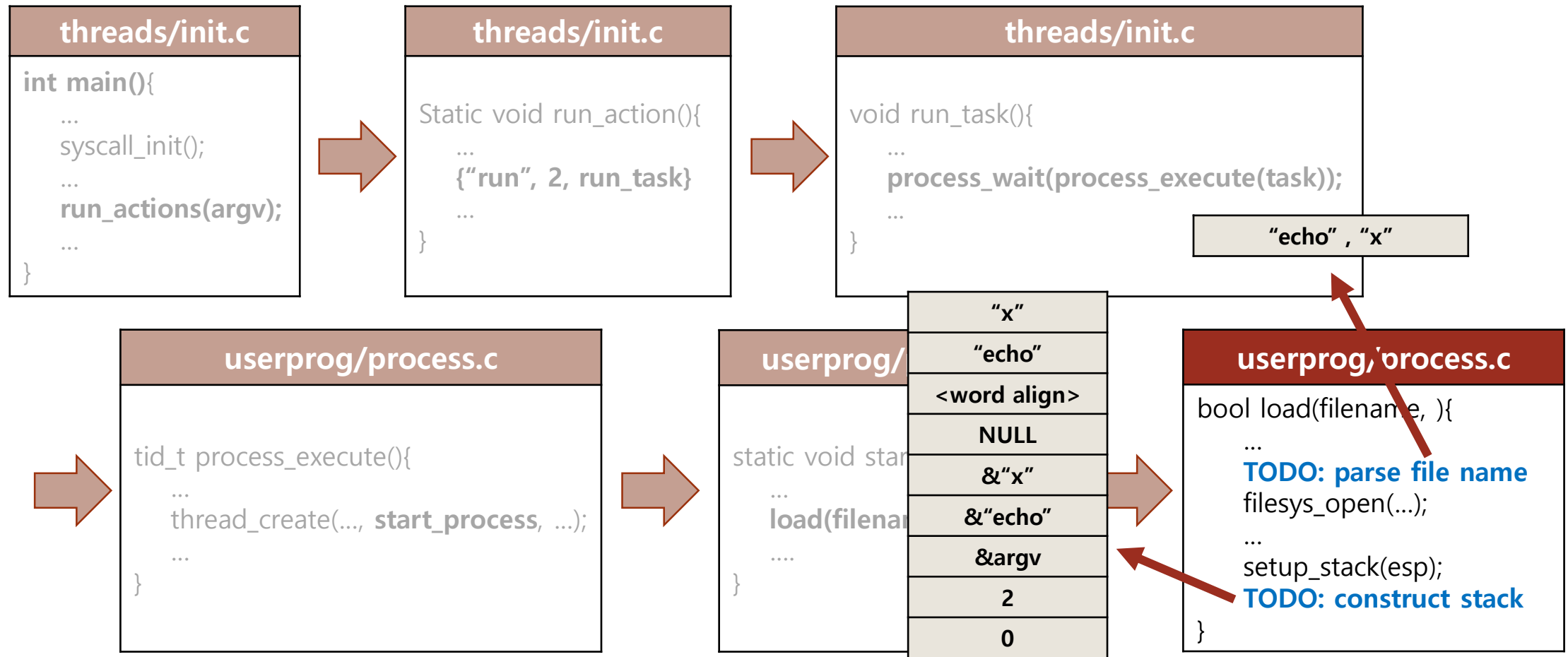
Code Level Flow

```
$ pintos --filesystem-size=2 -p ../examples/echo -a echo -- -f -q run 'echo x'
```



Code Level Flow

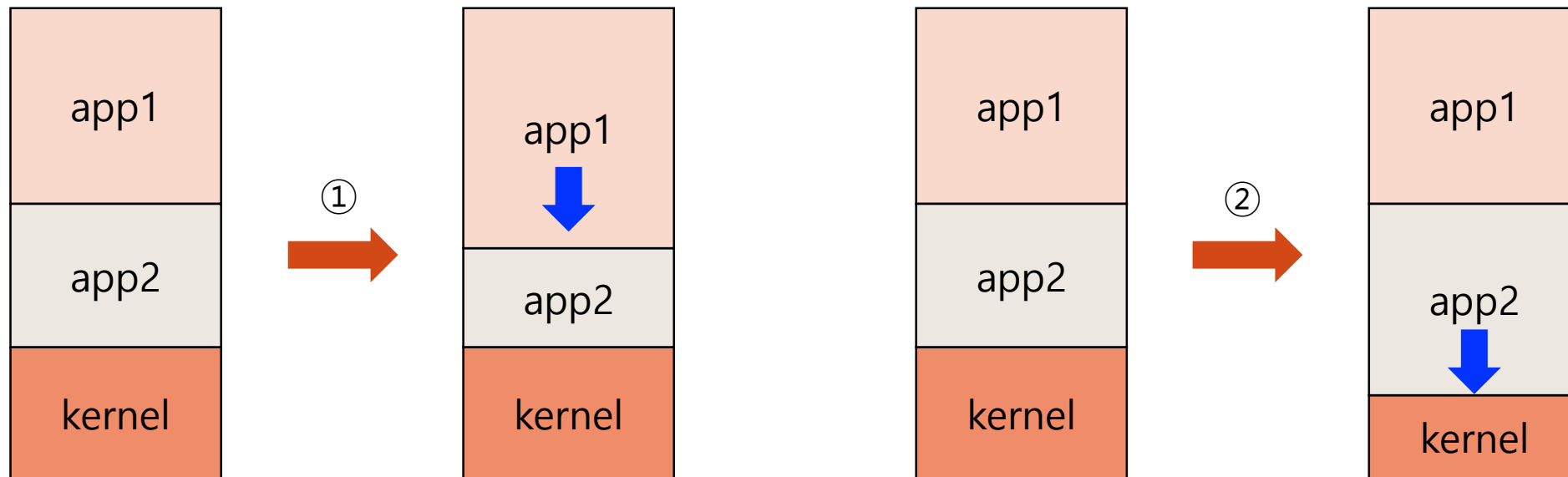
\$ pintos --fileysys-size=2 -p ../examples/echo -a echo -- -f -q run 'echo x'



Setup user stack based on 80x86 calling convention 13

Virtual Memory

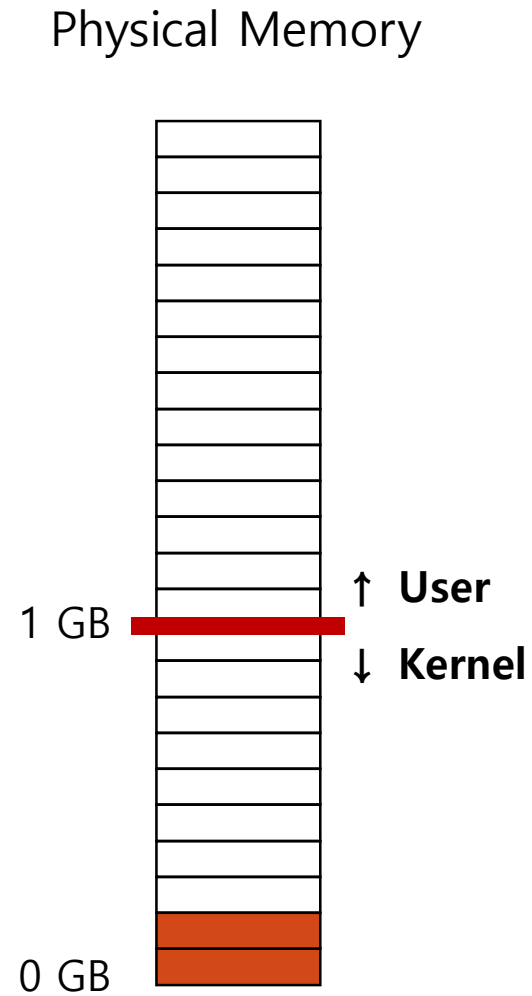
- Pintos divides memory into two region, user memory and kernel memory.
- If we use these memory areas directly, it's **hard to manage memory**.
- For example,
 - ① **each process can damage each other.**
 - ② **the process can corrupt kernel code** that is critical to running the operating system.



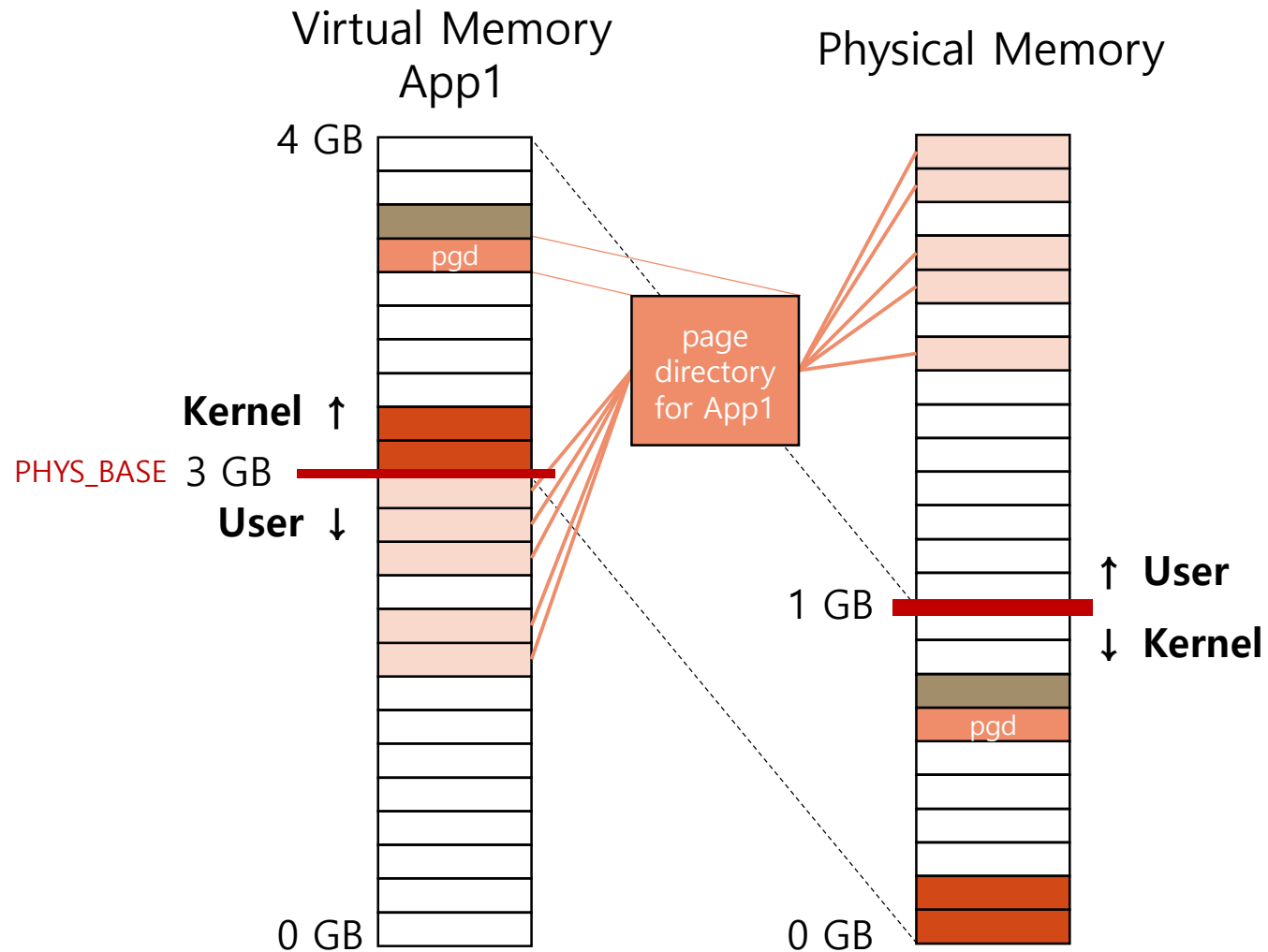
Virtual Memory

- To prevent these problems, operating systems adopt **virtual memory system**.
- Because of virtual memory, **each process can have its own memory area** and use it as if the process occupies the whole memory.
- **Pintos also manages memory regions by virtual memory.**
- Virtual memory is also divided into two regions: **user virtual memory** and **kernel virtual memory**.

Virtual Memory: Launch Application

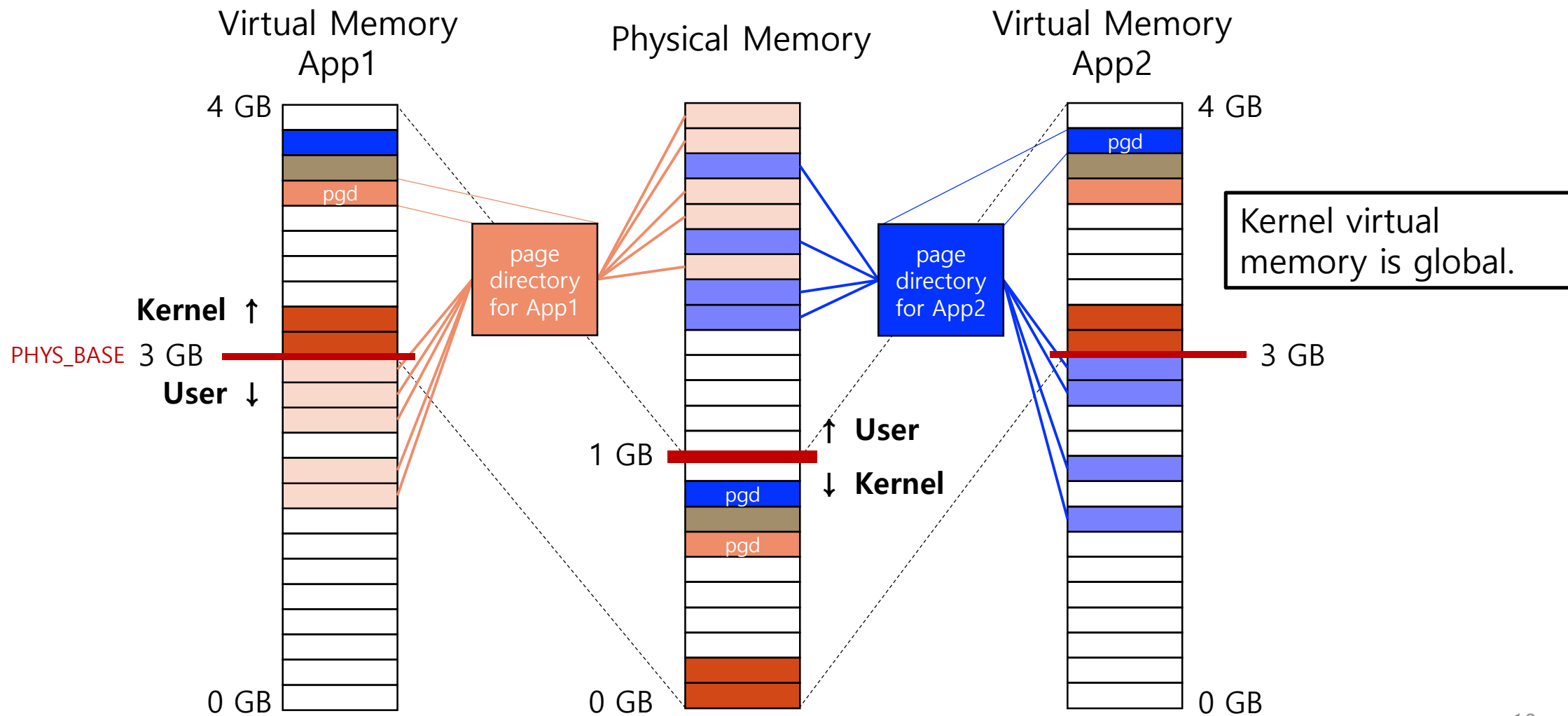


Virtual Memory: Launch Application



Kernel virtual memory is global.

Virtual Memory: Launch Application



Virtual Memory in Pintos

1. Each process has its own user virtual memory.
2. Pintos allocates 1 GB to kernel as global memory.
(PHYS_BASE (3 GB) ~ 4 GB in virtual memory)
3. Memory unit is a page in Pintos, which is size of 4 KB.
4. User program can access physical memory by translating virtual address via page directory and page table.
(Refer to A.7 'Page Table')

Virtual Memory

- Functions for page

- 1) `threads/vaddr.h`

- ✓ `is_user_vaddr()`, `is_kernel_vaddr()`

- Check that given virtual address is user/kernel virtual address

- ✓ `ptov()`, `vtop()`

- Translate physical address to kernel virtual address and vice versa

- 2) `threads/palloc.c`

- ✓ `palloc_get_page()`

- Get page from user/kernel memory pool

- 3) `userprog/pagedir.c`

- ✓ `pagedir_create()`

- Create page directory

- ✓ `pagedir_get_page()`

- Look up the physical address that corresponds to user virtual address in page directory

- ✓ `pagedir_set_page()`

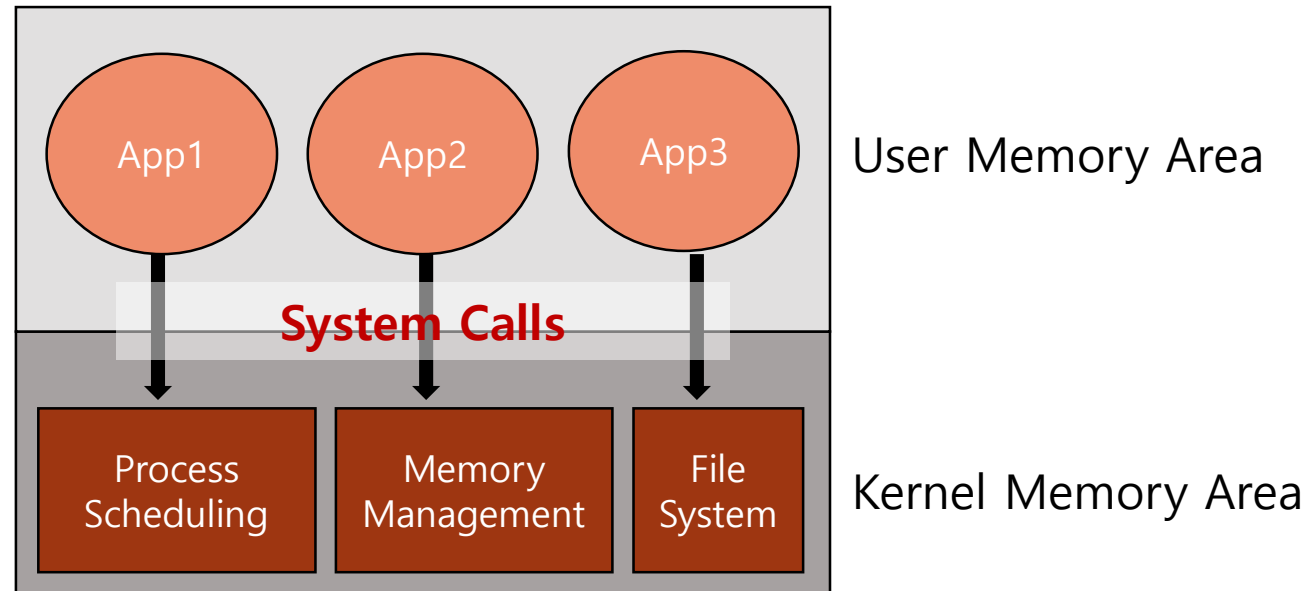
- Add mapping in page directory from user virtual address to the physical page

System Calls

- As we've seen, Pintos divides memory into user virtual memory and kernel virtual memory to protect each process and kernel code.
- Along with the concept of virtual memory, OS prevents user program from accessing the kernel memory which contains core functionalities.
- Then, how user program uses kernel's functionality?
- OS provides **system calls** to solve this problem.

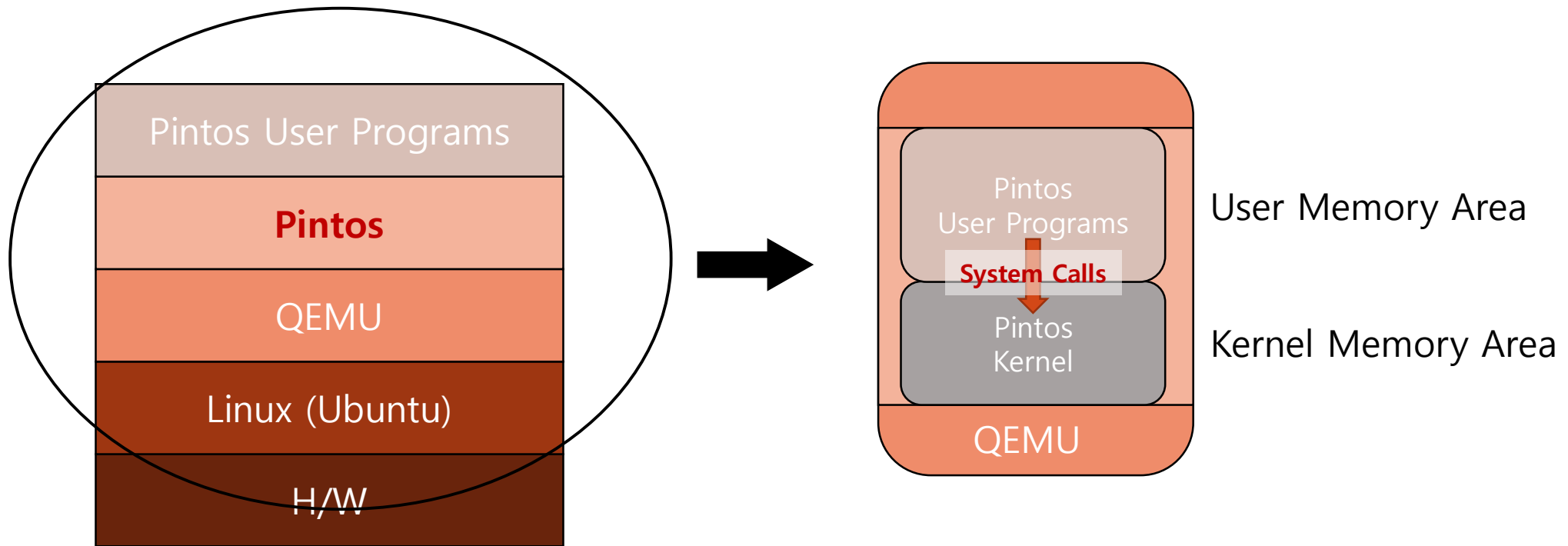
System Calls

- For safety, operating system provides two types of mode, user and kernel mode.
- **When user program is run in user mode, it can not access memory or disk.**
- These operations are performed in **kernel mode**.
- OS provides **system calls** to enter kernel mode.



System Calls

- Pintos provides **user level interface** of system calls in 'lib/user/syscall.c' and **skeleton of system call handler** in 'userprog/syscall.c'.



System Calls

- Procedure of system call in Pintos
 - User programs call system call function.

```
1 /* cat.c
2
3  Prints files specified on command line to the console. */
4
5 #include <stdio.h>
6 #include <syscall.h>
7
8 int
9 main (int argc, char *argv[])
10 {
11     bool success = true;
12     int i;
13
14     for (i = 1; i < argc; i++)
15     {
16         int fd = open (argv[i]);
17         if (fd < 0)
18         {
19             printf ("%s: open failed\n", argv[i]);
20             success = false;
21             continue;
22         }
```

open() system call

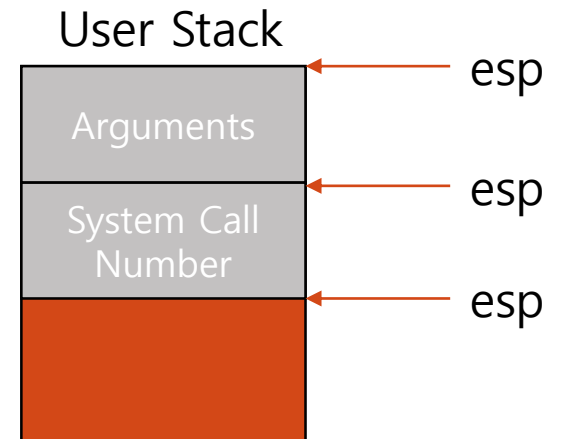
System Calls

- Procedure of system call in Pintos
 - **System call number** and additional **arguments** are pushed on caller's stack.
 - Invoke interrupt for system call by using '**int \$0x30**' instruction

```
102 int
103 open (const char *file)
104 {
105     return syscall1 (SYS_OPEN, file);
106 }
```

```
17 /* Invokes syscall NUMBER, passing argument ARG0, and returns the
18    return value as an 'int'. */
19 #define syscall1(NUMBER, ARG0)
20     ({
21         int retval;
22         asm volatile
23             ("pushl %[arg0]; pushl %[number]; int $0x30; addl $8, %%esp"
24              : "=a" (retval)
25              : [number] "i" (NUMBER),
26                [arg0] "g" (ARG0)
27              : "memory");
28         retval;
29     })
30
```

After returning from system call handler, restore stack pointer.



System Calls

- Procedure of system call in Pintos
 - Set the stack for interrupt and call interrupt handler

```
18 .func intr_entry
19 intr_entry:
20     /* Save caller's registers. */
21     pushl %ds
22     pushl %es
23     pushl %fs
24     pushl %gs
25     pushal
26
27     /* Set up kernel environment. */
28     cld          /* String instructions go upward. */
29     mov $SEL_KDSEG, %eax /* Initialize segment registers. */
30     mov %eax, %ds
31     mov %eax, %es
32     leal 56(%esp), %ebp /* Set up frame pointer. */
33
34     /* Call interrupt handler. */
35     pushl %esp
36     .globl intr_handler
37     call intr_handler
38     addl $4, %esp
39 .endfunc
```

Call interrupt handler

System Calls

- Procedure of system call in Pintos
 - `intr_handler()` calls system call handler.

```
344 void
345 intr_handler (struct intr_frame *frame)
346 {
347     bool external;
348     intr_handler_func *handler;
349
350     /* External interrupts are special.
351        We only handle one at a time (so interrupts must be off)
352        and they need to be acknowledged on the PIC (see below).
353        An external interrupt handler cannot sleep. */
354     external = frame->vec_no >= 0x20 && frame->vec_no < 0x30;
355     if (external)
356     {
357         ASSERT (intr_get_level () == INTR_OFF);
358         ASSERT (!intr_context ());
359
360         in_external_intr = true;
361         yield_on_return = false;
362     }
363
364     /* Invoke the interrupt's handler. */
365     handler = intr_handlers[frame->vec_no];
366     if (handler != NULL)
367         handler (frame);
```

Interrupt handler for **system call handler** has already been registered while Pintos was booting.*

※ Refer to the following function calls:

- 1) `main()` in 'threads/init.c' calls `syscall_init()` which is in 'userprog/syscall.c'
- 2) `syscall_init()` calls `intr_register_int()` in 'threads/interrupt.c'

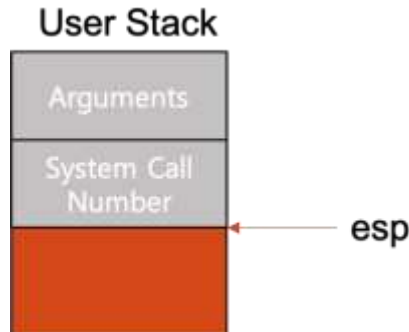
* source code: threads/interrupt.c

System Calls

- Procedure of system call in Pintos
 - `syscall_handler()` gets control, and it can access the stack via `'esp'` member of the `struct intr_frame` (in `threads/interrupt.h`).
 - 80x86 convention stores return value of system call in EAX register so that we can store the return value in `'eax'` member of the `struct intr_frame`.

```
15 static void
16 syscall_handler (struct intr_frame *f UNUSED)
17 {
18     printf ("system call!\n");
19     thread_exit ();
20 }
```

※ Pintos provides skeleton of system call handler.
We will develop this in this project!



```
20 struct intr_frame
21 {
22     /* Pushed by intr_entry in intr-stubs.S.
23        These are the interrupted task's saved registers. */
28     uint32_t ebx;          /* Saved EBX. */
29     uint32_t edx;          /* Saved EDX. */
30     uint32_t ecx;          /* Saved ECX. */
31     uint32_t eax;          /* Saved EAX. */
54     void *esp;             /* Saved stack pointer. */
55     uint16_t ss, :16;       /* Data segment for esp. */
56 };
```

* source code: `userprog/syscall.c`

What is File system?

```

:~$ ls -l
total 28
-rw-r--r-- 1 186 Nov 1 00:31 a.c
-rw-r--r-- 1 3319 Nov 1 00:32 a.txt
drwxr-xr-x 2 4096 Oct 31 02:54 Desktop
drwxr-xr-x 2 4096 Oct 31 02:54 Downloads
drwxr-xr-x 2 4096 Oct 31 02:58 example
drwxr-xr-x 2 4096 Oct 31 19:50 test
drwxr-xr-x 7 4096 Nov 1 00:30 tmp
    
```

ext4

User



```

#include <stdio.h>

int main(){
    .....

    FILE* fp = fopen("test/input.txt", "r");

    .....
}
    
```

Access

Requirements

Process Termination Messages

- When user program is terminated, kernel prints termination messages.
Output form is as follows:

Process Name: exit(exit status)\n

```
1  # -*- perl -*-
2  use strict;
3  use warnings;
4  use tests::tests;
5  check_expected ([<<'EOF']);
6  (exec-once) begin
7  (child-simple) run
8  child-simple: exit(81)
9  (exec-once) end
10 exec-once: exit(0)
11 EOF
```

<tests/userprog/exec-once.ck>

Refer to the following functions

threads/thread.c: thread_exit()
userprog/process.c: process_exit()

- Refer to Pintos manual 3.3.2

Process Termination Messages

- How can we get a process name?
 - Refer to struct thread

```
struct thread
{
    /* Owned by thread.c. */
    tid_t tid;
    enum thread_status status;
    char name[16];
}
```

Process Termination Messages

- How is user program terminated?
 - When ELF user program runs, `_start( )` in `lib/user/entry.c` is called at first.

```
void
_start (int argc, char *argv[])
{
    exit (main (argc, argv));
}
```

- After executing the program, `exit( )` system call is called.
- Pintos only provides `exit()` system call API, but the `exit()` system call API has not yet been implemented.

Process Termination Messages

- How is the user program terminated?
 - Flow of function calls
 - exit() in lib/user/syscall.c
 - > syscall1 (SYS_EXIT, status) in lib/user/syscall.c
 - > syscall_handler() in userprog/syscall.c
 - > thread_exit() in threads/thread.c
 - > process_exit() in userprog/process.c

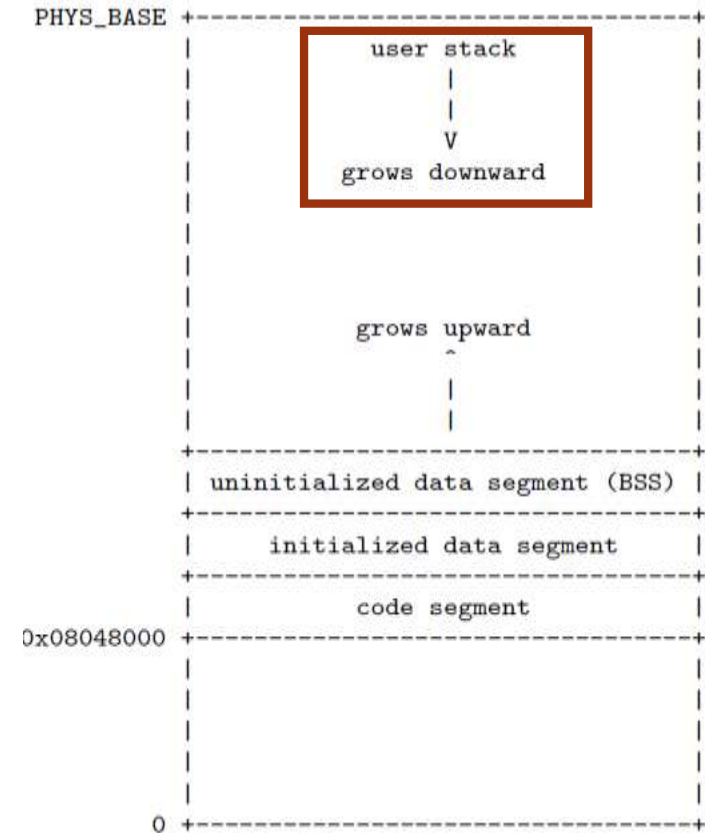
Refer to slide pg. 25-29

Argument Passing

1. User program can have multiple arguments.

```
~ /bin/ls -l foo bar  
-rw-r--r-- 1 root root 0 Sep 11 02:59 bar  
-rw-r--r-- 1 root root 0 Sep 11 02:58 foo
```

2. **Parse the arguments and allocate it to memory according to 80x86 calling convention**
 - Refer to the next slides and Pintos manual 3.5
3. Assume that the length of arguments is less than 4 KB
 - Test programs use less than 128 Bytes as arguments.



<User virtual memory in Pintos>

Argument Passing

- `"/bin/ls -l foo bar"` will be parsed into `"/bin/ls"`, `"-l"`, `"foo"`, `"bar"`

Argument Passing

- `"/bin/ls -l foo bar"` will be parsed into `"/bin/ls"`, `"-l"`, `"foo"`, `"bar"`

0xC0000000 (PHYS_BASE)

- 0x00000004

0xBFFFFFFC

Address	Name	Data	Type
0xbfffffffcc	argv[3][...]	'bar\0'	char[4]
0xbfffffff8	argv[2][...]	'foo\0'	char[4]
0xbfffffff5	argv[1][...]	'-l\0'	char[3]
0xbffffffed	argv[0][...]	'/bin/ls\0'	char[8]
0xbffffffec	word-align	0	uint8_t
0xbffffffe8	argv[4]	0	char *
0xbffffffe4	argv[3]	0xbfffffffcc	char *
0xbffffffe0	argv[2]	0xbfffffff8	char *
0xbffffffdc	argv[1]	0xbfffffff5	char *
0xbffffffd8	argv[0]	0xbffffffed	char *
0xbffffffd4	argv	0xbffffffd8	char **
0xbffffffd0	argc	4	int
0xbffffffcc	return address	0	void (*) ()

- You can start implementation of argument passing after the following function.
 - ✓ `userprog/process.c` : static bool **setup_stack**(void **esp)
 - ✓ Refer to '[Code Level Flow](#)' in the previous chapter

Argument Passing

- `"/bin/ls -l foo bar"` will be parsed into `"/bin/ls"`, `"-l"`, `"foo"`, `"bar"`
 - ✓ Push arguments at the top of the stack

Address	Name	Data	Type
0xbfffffffcc	argv[3][...]	'bar\0'	char[4]
0xbfffffff8	argv[2][...]	'foo\0'	char[4]
0xbfffffff5	argv[1][...]	'-l\0'	char[3]
0xbffffffed	argv[0][...]	'/bin/ls\0'	char[8]
0xbffffffec	word-align	0	uint8_t
0xbffffffe8	argv[4]	0	char *
0xbffffffe4	argv[3]	0xbfffffffcc	char *
0xbffffffe0	argv[2]	0xbfffffff8	char *
0xbffffffdc	argv[1]	0xbfffffff5	char *
0xbffffffd8	argv[0]	0xbffffffed	char *
0xbffffffd4	argv	0xbffffffd8	char **
0xbffffffd0	argc	4	int
0xbffffffcc	return address	0	void (*) ()

Argument Passing

- `"/bin/ls -l foo bar"` will be parsed into `"/bin/ls"`, `"-l"`, `"foo"`, `"bar"`
 - ✓ Push address of each argument

Address	Name	Data	Type
0xbfffffffcc	argv[3][...]	'bar\0'	char[4]
0xbfffffff8	argv[2][...]	'foo\0'	char[4]
0xbffffff5	argv[1][...]	'-l\0'	char[3]
0xbffffffed	argv[0][...]	'/bin/ls\0'	char[8]
0xbffffffec	word-align	0	uint8_t
0xbffffffe8	argv[4]	0	char *
0xbffffffe4	argv[3]	0xbfffffffcc	char *
0xbffffffe0	argv[2]	0xbffffff8	char *
0xbffffffdc	argv[1]	0xbffffff5	char *
0xbffffffd8	argv[0]	0xbffffffed	char *
0xbffffffd4	argv	0xbffffffd8	char **
0xbffffffd0	argc	4	int
0xbffffffcc	return address	0	void (*) ()

NULL pointer sentinel
(required by C standard)

Argument Passing

- `"/bin/ls -l foo bar"` will be parsed into `"/bin/ls"`, `"-l"`, `"foo"`, `"bar"`
 - ✓ result of **hex_dump()** : This function is very useful for debug (in `src/lib/stdio.c`).

```

bfffffff00 00 00 00 00 | .....|
bffffffd0 04 00 00 00 d8 ff ff bf-ed ff ff bf f5 ff ff bf |.....|
bffffffe0 f8 ff ff bf fc ff ff bf-00 00 00 00 00 2f 62 69 |...../bi|
bfffffff0 6e 2f 6c 73 00 2d 6c 00-66 6f 6f 00 62 61 72 00 |n/ls.-l.foo.bar.|

```

↓ ↓ ↓ ↑
0xbfffffff8 0xbfffffff8 foo\0 bar\0
Top of the stack

Address	Name	Data	Type
0xbffffffc	argv[3][...]	'bar\0'	char[4]
0xbffffff8	argv[2][...]	'foo\0'	char[4]
0xbffffff5	argv[1][...]	'-l\0'	char[3]
0xbffffffed	argv[0][...]	'/bin/ls\0'	char[8]
0xbffffffec	word-align	0	uint8_t
0xbffffffe8	argv[4]	0	char *
0xbffffffe4	argv[3]	0xbfffffff8	char *
0xbffffffe0	argv[2]	0xbfffffff8	char *
0xbffffffdc	argv[1]	0xbffffff5	char *
0xbffffffd8	argv[0]	0xbffffffed	char *
0xbffffffd4	argv	0xbffffffd8	char **
0xbffffffd0	argc	4	int
0xbffffffcc	return address	0	void (*)()

Argument Passing

- In `userprog/process.c`, there is `setup_stack( )` which allocates a minimal stack page (4KB).
- Since the given code only allocates stack page, we need to make up the stack after `setup_stack( )`.
- Make up the stack referring to "**3.5 80x86 Calling Convention**" in Pintos manual

```
304  /* Set up stack. */
305  if (!setup_stack (esp))
306      goto done;
307
308  /* Start address. */
309  *eip = (void (*) (void)) ehdr.e_entry;
```

Write codes here!

System Calls

1. Students will have to implement the following system calls
(**Requirements** of each system call are described in **Pintos manual 3.3.4.**)

[Prj1-1]

- **halt, exit, exec, wait, read, write**
(※ Pintos exec is different from UNIX exec)
- Two new system calls (**fibonacci, max_of_four_int**)

[Prj1-2]

- **create, remove, open, close, filesize, seek, tell, read, write**
(full implementation)

2. **read / write** are implemented in two stages

- Prj1-1: standard input/output only (STDIN=0, STDOUT=1)
- Prj1-2: extend to files using file descriptor

System Calls: General System Calls

- `halt()`
 - 1) Terminate Pintos by calling `shutdown_power_off()`
- `exit()`
 - 1) Terminate the current user program, returning status to the kernel

System Calls: General System Calls

- `exec()`
 - 1) Create child process
 - 2) Refer to `process_execute()` in `userprog/process.c`
- `wait()`
 - 1) What `wait()` system call should do is **wait child process until it finishes its work.**
 - 2) Check that child thread ID is valid
 - 3) Get the exit status from child thread when the child thread is dead
 - 4) To prevent termination of process before return from `wait()`, you can use **busy waiting technique*** or `thread_yield()` in `threads/thread.c`.

System Calls: General System Calls

- write() and read()
 - Not full implementation, but at least read from STDIN, write to STDOUT.
- 1) File Descriptor of STDIN, STDOUT
 - ✓ STDIN = 0, STDOUT = 1
- 2) Use the following function to implement read(0)
 - ✓ pintos/src/devices/input.c: uint8_t **input_getc**(void)
- 3) Use the following function to implement write(1)
 - ✓ pintos/src/lib/kernel/console.c: void **putbuf**(...)

System Calls: Code Level Flow

- Start from `main()` in `threads/init.c`
- `run_actions (argv);` will be invoked.

```
/* Run actions specific to this thread */  
run_actions (argv);  
  
/* Finish up. */  
shutdown ();  
thread_exit ();
```

System Calls: Code Level Flow

- Focus on {"run", 2, run_task}
- **a->function (argv);** invokes **run_task()**.

```
static void
run_actions(char **argv)
{
    /* An action. */
    struct action
    {
        char *name;
        int argc;
        void (*function)(char **argv);
    };

    /* Table of supported actions. */
    static const struct action actions[] =
    {
        {"run", 2, run_task,
#ifdef FILESYS
        {"ls", 1, fsutil_ls,
```

```
while (*argv != NULL)
{
    const struct action *a;
    int i;

    /* Find action name. */
    for (a = actions; ; a++)
        if (a->name == NULL)
            PANIC ("unknown action '%s' (use
else if (!strcmp (*argv, a->name))
            break;

    /* Check for required arguments. */
    for (i = 1; i < a->argc; i++)
        if (argv[i] == NULL)
            PANIC ("action '%s' requires %d a

    /* Invoke action and advance. */
    a->function(argv);
    argv += a->argc;
}
```

System Calls: Code Level Flow

- `run_task()` invokes `process_execute()`.

```
/* Runs the task specified in ARGV[1]. */
static void
run_task (char **argv)
{
    const char *task = argv[1];

    printf ("Executing '%s':\n", task);
#ifdef USERPROG
    process_wait (process_execute (task));
#else
    run_test (task);
#endif
    printf ("Execution of '%s' complete.\n", task);
}
```

What does 'task' contain?
Does it contain only file name?
Refer to printf statement
`strtok_r()` in lib/string.c will help you.

System Calls: Code Level Flow

- **thread_create()** will enroll user program name.
- And, it also enrolls function, **start_process()**, which will launch user program.

```
tid_t
process_execute (const char *file_name)
{
    char *fn_copy;
    tid_t tid;

    /* Make a copy of FILE_NAME.
       Otherwise there's a race between the caller and load(). */
    fn_copy = palloc_get_page (0);
    if (fn_copy == NULL)
        return TID_ERROR;
    strcpy (fn_copy, file_name, PGSIZE);

    /* Create a new thread to execute FILE_NAME. */
    tid = thread_create (file_name, PRI_DEFAULT, start_process, fn_copy);
    if (tid == TID_ERROR)
        palloc_free_page (fn_copy);
    return tid;
}
```

System Calls: Code Level Flow

- When process scheduling is invoked, the child process (user program) will be executed by wrapper function of `_start()` in lib/user/entry.c.

```
void
_start (int argc, char *argv[])
{
    exit (main (argc, argv));
}
```

System Calls: Code Level Flow

- When ELF executable (user program) is finished, `exit()` system call is called.
- After `exit()` system call, it returns to `process_wait()`.

System Calls: Code Level Flow

- Parent process which ran `process_execute()` should be waiting in `process_wait()` until the child process is finished.

```
/* Runs the task specified in ARGV[1]. */
static void
run_task (char **argv)
{
    const char *task = argv[1];

    printf ("Executing '%s':\n", task);
#ifdef USERPROG
    process_wait (process_execute (task));
#else
    run_test (task);
#endif
    printf ("Execution of '%s' complete.\n", task);
}
```

System Calls: Source Codes

1. lib/user/syscall.h and lib/user/syscall.c
 - APIs for system calls are already given in Pintos code.
 - **You don't have to add something for system call APIs.**
2. userprog/syscall.h
 - There is only one prototype `syscall_init()` which registers system call interrupts when Pintos was booted.
 - You can write prototype of system calls in this file.
3. userprog/syscall.c
 - You must make **`syscall_handler()`** handle system calls.
 - If you have done argument passing, you can get system call number from **`intr_frame *f`**.
 - **`esp`** member of **`intr_frame *f`** points to system call number.
(You can refer to **`lib/syscall-nr.h`** to check each system call number)
 - And then you can use `switch` statement to classify system calls.
(What really these system calls do would be written here.)

Additional System Calls

- **Implement new system calls into Pintos**

1. **int fibonacci(int n)**

- ✓ Return N th value of Fibonacci sequence

2. **int max_of_four_int(int a, int b, int c, int d)**

- ✓ Return the maximum of a, b, c and d

※ Use 'fibonacci' and 'max_of_four_int' as the name of new system calls.

Don't use other name to implement new system calls above.

Additional System Calls

- **Write user level program which uses new system calls**

1. Make additional.c in pintos/src/examples
2. Write simple example by using new system calls

- 3. Name of execution file should be 'additional'**

4. Usage : ./additional [num 1] [num 2] [num 3] [num 4]

Function : Print the result of 'fibonacci' system call using [num 1] as parameter

Print the result of 'max_of_four_int' system call using [num 1, 2, 3, 4] as parameter

Example :

```
$ ./additional 10 20 62 40
55 62
```

5. Run the following command to check your program works properly.

```
pintos/src/userprog$ pintos --fileys-size=2 -p ../examples/additional -a additional -- -f -q run
'additional 10 20 62 40'
```

'additional' should be run on Pintos, not CSPRO server.

Additional System Calls

1. How to identify system call number?
 - ✓ Refer to 'lib/syscall-nr.h'
2. How to return system call's result?
 - ✓ Check argument 'struct intr_frame' of **syscall_handler()** in syscall.c
 - ✓ Refer to 'System Calls' in Prerequisites

Additional System Calls

- To compile newly added user program, "additional", you need to modify **Makefile** in **src/examples**.
- Refer to how other user programs are written in **Makefile**

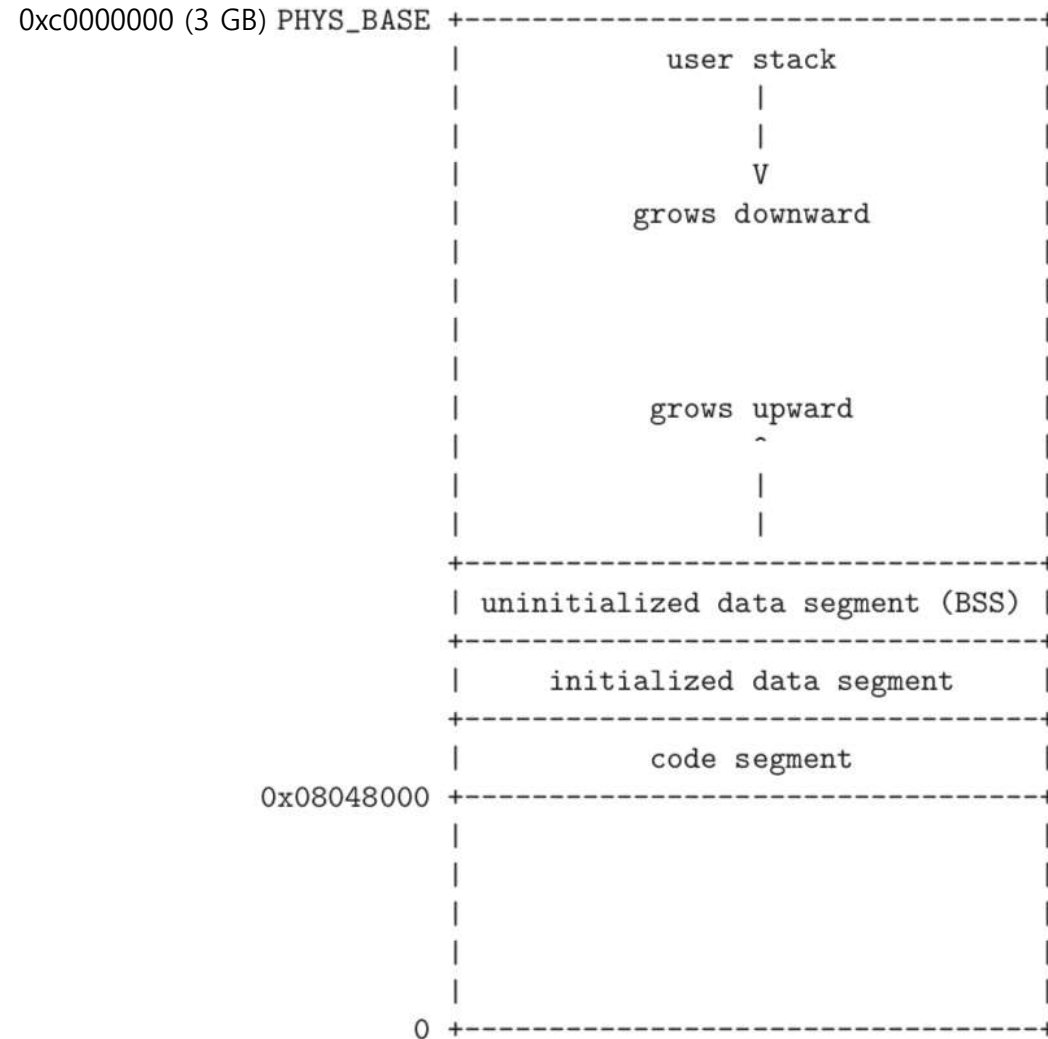
Additional System Calls: Source Codes

1. lib/user/syscall.h
 - Write prototype of 2 new system call APIs
2. lib/user/syscall.c
 - Define new **syscall4()** function for **max_of_four_int()** (lib/user/syscall.c)
 - Define **fibonacci()** and **max_of_four_int()** system calls APIs
3. lib/syscall-nr.h
 - Add system call numbers for 2 new system calls
4. userprog/syscall.h
 - Write prototype of 2 new system calls
5. userprog/syscall.c
 - Define **fibonacci()** and **max_of_four_int()** system calls
 - What really these system calls do would be written here.

Accessing User Memory

1. User program can pass an invalid pointer.
 - **NULL** pointer such as **open (NULL);** in tests/userprog/open-null.c
 - **Unmapped** virtual memory
 - Pointer to **kernel** address space
- 2. Invalid pointers must be rejected** without harm to kernel or other running process.
3. It can be implemented in 2 ways:
 - 1) Verify the **validity of a user-provided pointer**, then dereference it.
 - 2) Check only that a user pointer points below PHYS_BASE, then dereference it.
If the pointer is invalid, it will cause a "page fault". You can handle it by modifying the code **page_fault()** in 'userprog/exception.c'
4. Refer to Pintos manual 3.1.5

Accessing User Memory



Refer to Pintos manual 3.1.4

Mapped User Virtual Memory

Unmapped User Virtual Memory

Accessing User Memory

- To verify the validity of a user-provided pointer, you can use functions in [userprog/pagedir.c](#) and [threads/vaddr.h](#).
- Check **Unmapped** virtual memory using `pagedir_get_page( )`
- Check pointer to **kernel** address space using `is_user_vaddr( )` and `is_kernel_vaddr( )`
- Use these functions to verify the validity of given pointer

Base File System

Base File System

- **To implement system call about file system, we need to understand base file system of Pintos**
 - We need to interface to the file system code. So, we need to know **API usage**.
 - Pintos provides simple but complete file system.
(Look `fileSYS.h` and `file.h` at `fileSYS/`)
 - Don't need to understand whole structure of file system.
 - No need to modify the file system code for this project.

Base File System

- **Limitation of Base File System (See also p.23-24):**
 - No internal synchronization
 - File size is fixed at creation time
 - File data is allocated as a contiguous range of sectors on disks
 - No subdirectories
 - File name length limitation (up to 14 characters)

Base File System

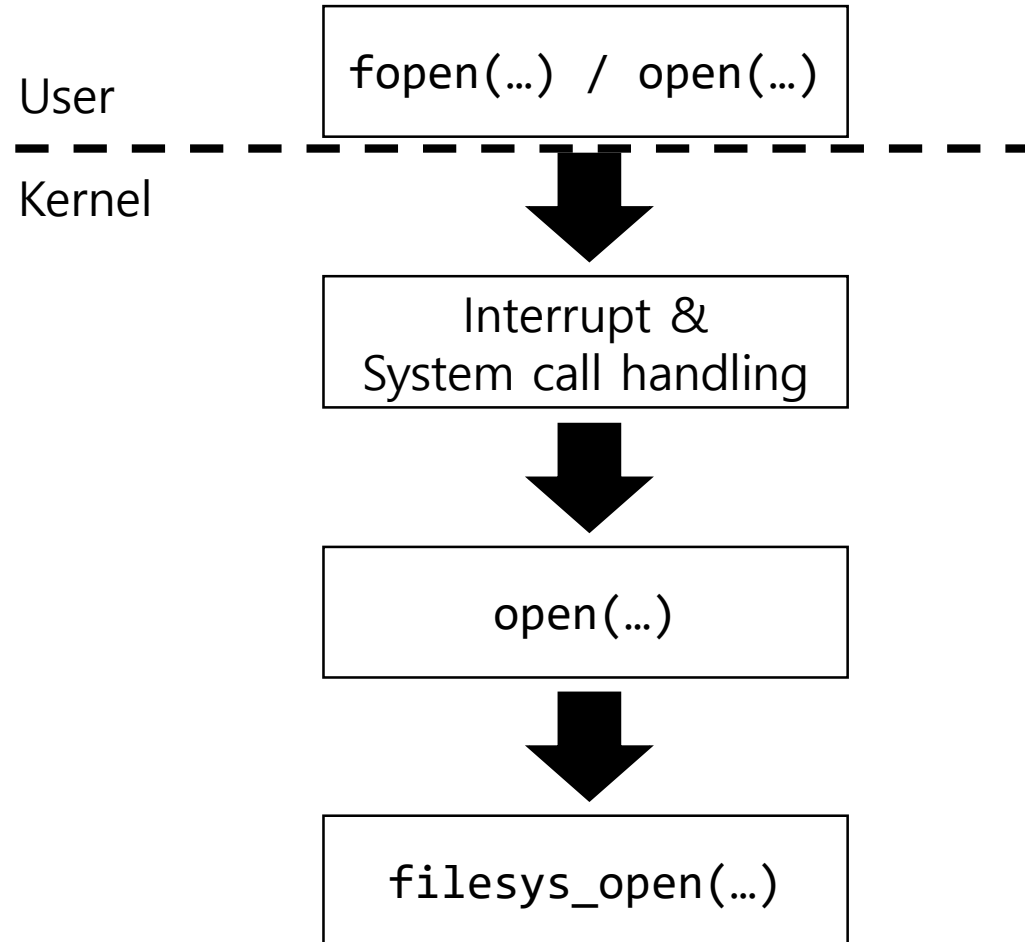
- Usage about file system on project 1
 - There is no sub-directories.
 - We assume that we work on root directory (/) of file system.
- File access at Kernel
 - Can access through struct file (See filesystems/file.c)
 - (At this project, it doesn't need to access on inode level)

```
/* An open file. */
struct file
{
    struct inode *inode;          /* File's inode. */
    off_t pos;                   /* Current position. */
    bool deny_write;             /* Has file_deny_write() been called? */
};
```

File System Call Implementation

File System Call Implementation

- Basic file system usage interfaces:



- Call fopen(or open) at user program

- Interrupt 0x30 and call system call handler

We need to implement

- Call open system call

- At internal implementation of open, it calls `filesys_open` function and open that file

File System Call Implementation

- Example in Pintos

```
/* Open a file. */  
  
#include <syscall.h>  
#include "tests/lib.h"  
#include "tests/main.h"  
  
void  
test_main (void)  
{  
    int handle = open ("sample.txt");  
    if (handle < 2)  
        fail ("open() returned %d", handle);  
}
```

```
int  
open(const char *file)  
{  
    .....  
    struct file * f = filesys_open(...);  
    .....  
}
```

```
struct file *  
filesys_open (const char *name)  
{  
    struct dir *dir = dir_open_root ();  
    struct inode *inode = NULL;  
    if (dir != NULL)  
        dir_lookup (dir, name, &inode);  
    dir_close (dir);  
  
    return file_open (inode);  
}
```

System call acts like interfaces for calling file_xxx

File System Call Implementation

- File Descriptor (Similar to FILE * of standard C)
 - Return value of open, create
- At Pintos, each thread get and manage independent file descriptor.
- File descriptor of STDIN, STDOUT, STDERR
 - STDIN = 0
 - STDOUT = 1
 - (At Linux or other OSes, STDERR = 2)

File System Call Implementation

- read and write with stdin/stdout
 - Read from stdin – read(0)
 - Use `uint8_t` `input_getc(void)` (`devices/input.c`)
 - Write to stdout – write(1)
 - Use `void` `putbuf(...)` (`lib/kernel/console.c`)

Useful APIs

- filesystems/filesys.h

```
bool filesystems_create (const char *name, off_t initial_size);
struct file *filesystems_open (const char *name);
bool filesystems_remove (const char *name);
```

- filesystems/file.h

```
/* Opening and closing files. */
struct file *file_open (struct inode *);
void file_close (struct file *);

/* Reading and writing. */
off_t file_read (struct file *, void *, off_t);
off_t file_write (struct file *, const void *, off_t);

/* Preventing writes. */
void file_deny_write (struct file *);
void file_allow_write (struct file *);

/* File position. */
void file_seek (struct file *, off_t);
off_t file_tell (struct file *);
off_t file_length (struct file *);
```

Kernel Functions

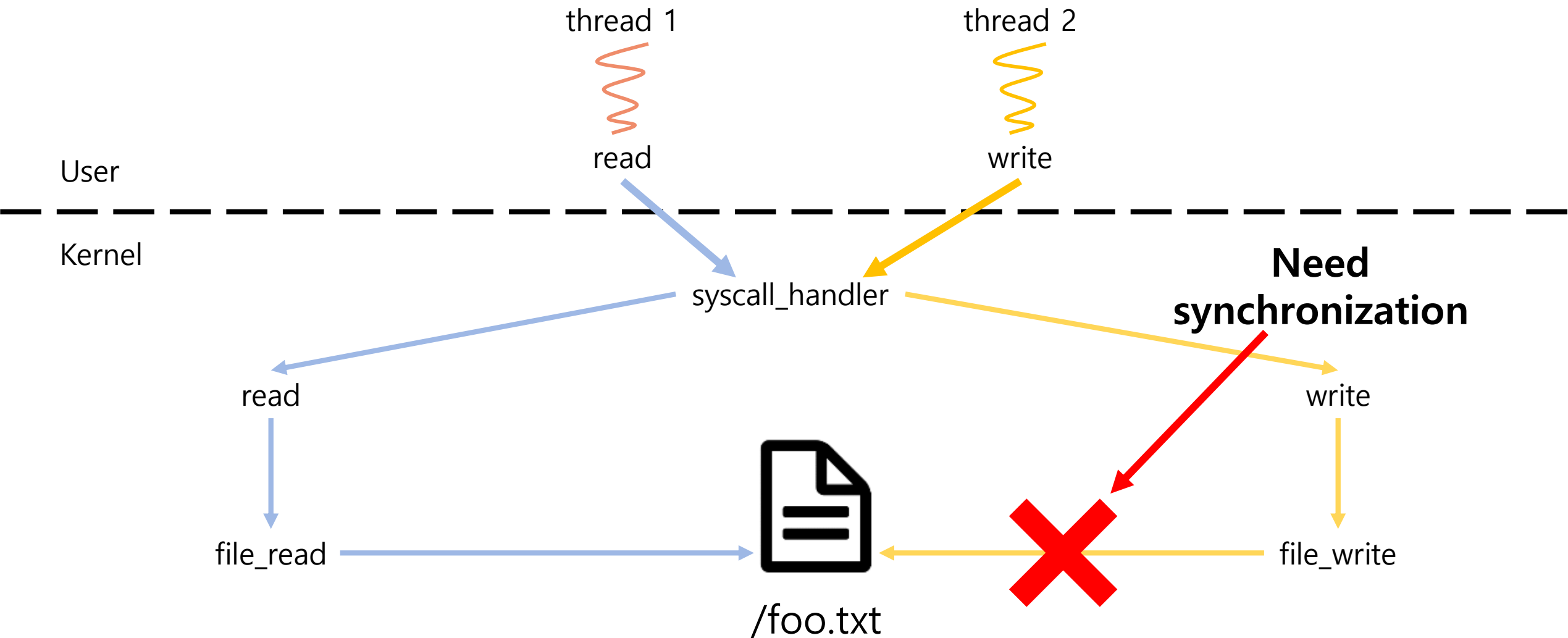
Denying Writes to Executable files

- Process is executed after load to memory.
 - So, removing executable file may not be a problem after executed.
- But Pintos doesn't want to delete executable file of running program.
- These file system function may be useful for this problem:
 - `void file_deny_write(struct file *)` (filesys/file.c)
 - `void file_allow_write(struct file *)` (filesys/file.c)

Protect Critical Section

- Critical Section
 - N Processes all competing to use some shared data
 - Each Process has a code segment, called **critical section**, in which the **shared data is accessed**
 - Problem: Ensure that when one process is executing in its critical section, **no other process can execute** in its critical section
- Using test case of Pintos, check code segment that should be considered as critical section
 - Protect this code segment using synchronization APIs.
 - ex) syn-read, syn-write

Critical Section – Example in Pintos



Synchronization APIs (See also p.67-68)

- Lock

```
/* Lock. */
struct lock
{
    struct thread *holder;    /* Thread holding lock (for debugging). */
    struct semaphore semaphore; /* Binary semaphore controlling access. */
};

void lock_init (struct lock *);
void lock_acquire (struct lock *);
bool lock_try_acquire (struct lock *);
void lock_release (struct lock *);
bool lock_held_by_current_thread (const struct lock *);
```

Synchronization APIs (See also p.67-68)

- Semaphore

```
/* A counting semaphore. */
struct semaphore
{
    unsigned value;           /* Current value. */
    struct list waiters;      /* List of waiting threads. */
};

void sema_init (struct semaphore *, unsigned value);
void sema_down (struct semaphore *);
bool sema_try_down (struct semaphore *);
void sema_up (struct semaphore *);
void sema_self_test (void);
```

Suggested Order of Implementation (1)

Refer to Pintos manual 3.2

- You can not see the result until you implement core functionalities.
- Students are highly encouraged to read/understand these slides and Pintos manual first, design the structures and then start to implement.

- 1) **Argument Passing**: After implementing it, check the result using `hex_dump()`.
 - 2) **User Memory Access**: Protect user memory accesses from system calls.
 - 3) **System Call Handler**: Implement `syscall_handler()` to handle system call.
 - 4) **System Call Implementation**: Implement **`exec()`, `exit()`, `write()`, `read()`** first, and then others.
 - 5) **Additional Implementation**: Implement `fibonacci()`, `max_of_four_int()`.
- ※ Refer to source codes in `src/tests/userprog`

Suggested Order of Implementation (1)

Refer to Pintos manual 3.2

- You can not see the result until you implement core functionalities.
- Students are highly encouraged to read/understand these slides and Pintos manual first, design the structures and then start to implement.

- 1) Argument Passing:** After implementing it, check the result using `hex_dump()`
- 2) User Memory Access:** Make a plan for protecting user memory accesses from system calls
- 3) System Call Handler:** Implement `syscall_handler()` to handle system call
- 4) System Call Implementation:** Implement `exec()`, `exit()`, `write()`, `read()` first
- 5) Additional Implementation:** Implement...

※ Refer to source codes in `src/tests/userprog`

Refer to Code Level Flow

`src/userprog/process.c : load()`

Check parameters of `load()`

If you want to check the dump values before implementing `process_wait()`, insert infinite loop in `process_wait()` to block process
(You should finish to implement `process_wait()` later)

Suggested Order of Implementation (1)

Refer to Pintos manual 3.2

- You can not see the result until you implement core functionalities.
- Students are highly encouraged to read/understand these slides and Pintos manual first, design the structures and then start to implement.

- 1) **Argument Passing:** After implementing it, check the result using `hex_dump()`
- 2) **User Memory Access:** Make a plan for protecting user memory accesses from system calls
- 3) **System Call Handler:** Implement `syscall_handler()` to handle system call
- 4) **System Call Implementation:** Implement `exec()`, `exit()`, `write()`, `read()` first
- 5) **Additional Implementation:** Implement `fibonacci()`, `max_of_four_int()`

※ Refer to source codes in `src/tests/userprog`

Refer to `src/threads/vaddr.h`

Recommend to implement the function which checks the validity of given address

Suggested Order of Implementation (1)

Refer to Pintos manual 3.2

- You can not see the result until you implement core functionalities.
- Students are highly encouraged to read/understand these slides and Pintos manual first, design the structures and then start to implement.

- 1) **Argument Passing:** After implementing it, check the result using `hex_dump()`
- 2) **User Memory Access:** Make a plan for protecting user memory accesses from system calls
- 3) **System Call Handler:** Implement `syscall_handler()` to handle system call
- 4) **System Call Implementation:** Implement `exec()`, `exit()`, `write()`, `read()` first
- 5) **Additional Implementation:** Implement `fibonacci()`, `max_of_four_int()`

※ Refer to source codes in `src/tests/userprog`

src/userprog/syscall.c : syscall_handler()

Check argument 'struct intr_frame' of `syscall_handler()` in `syscall.c`
(struct `intr_frame` is in `src/threads/interrupt.h`)

Suggested Order of Implementation (1)

Refer to Pintos manual 3.2

- You can not see the result until you implement core functionalities.
- Students are highly encouraged to read/understand these slides and Pintos manual first, design the structures and then start to implement.

- 1) **Argument Passing:** After implementing it, check the result using `hex_dump()`
- 2) **User Memory Access:** Make a plan for protecting user memory accesses from system calls
- 3) **System Call Handler:** Implement `syscall_handler()` to handle system call
- 4) **System Call Implementation:** Implement `exec()`, `exit()`, `write()`, `read()` first
- 5) **Additional Implementation:** Implement `fibonacci()`, `max_of_four_int()`

※ Refer to source codes in `src/tests/userprog`

**Synchronization will be needed
(You can use busy waiting)
exit status is -1 when syscall_handler is terminated in
abnormal way**

Suggested Order of Implementation (1)

Refer to Pintos manual 3.2

- You can not see the result until you implement core functionalities.
- Students are highly encouraged to read/understand these slides and Pintos manual first, design the structures and then start to implement.

- 1) **Argument Passing:** After implementing it, check the result using `hex_dump()`
- 2) **User Memory Access:** Make a plan for protecting user memory accesses from system calls
- 3) **System Call Handler:** Implement `syscall_handler()` to handle system call
- 4) **System Call Implementation:** Implement `exec()`, `exit()`, `write()`, `read()` first
- 5) **Additional Implementation:** Implement `fibonacci()`, `max_of_four_int()`

※ Refer to source codes in `src/tests/userprog`

Modify the followings:
`src/lib/syscall-nr.h`
`src/lib/syscall.h`
`src/lib/syscall.c`

Suggested Order of Implementation (2)

- **Implement system calls about file system**
 - System calls
 - create, remove, open, close, filesize, read, write, seek, tell
 - See Pintos manual p.29~32 for system call implementation.

1. All 76 tests in this project will be graded.
 1. Prj1-1: 21 tests / Prj1-2: 55 tests
(Refer to the test case lists in the next slides)
(※ Rows **Highlighted in yellow** in the tables are the 21 tests for Prj1-1)
2. Total score is 100 which consists of 80 for test cases and 20 for documentation.
3. Additional 2.5 points for each additional system call implementation.
(5 points for **fibonacci()** and **max_of_four_int()** in implementation)
※ It will be calculated in development part (80%), so the total point will be 4 points (5*80%).
4. Grading script (**make grade** or **make check** in src/userprog) provided by Pintos will be used.
5. Refer to '**grade**' and '**results**' files in src/userprog/build after grading
(**'grade'** file is only created when you use **make grade**)

6. Test cases are classified
in functionality / robustness / no-vm / base file system (filesys base)
7. Refer to the followings for checking each test case's point based on the test type
 - `pintos/src/tests/userprog/Rubric.functionality`
 - `pintos/src/tests/userprog/Rubric.robustness`
 - `pintos/src/tests/userprog/no-vm/Rubric`
 - `pintos/src/tests/filesys/base/Rubric`

Evaluation

Functionality		
No.	Name	Point
1	args-none	3
2	args-single	3
3	args-multiple	3
4	args-many	3
5	args-dbl-space	3
6	exec-once	5
7	exec-multiple	5
8	exec-arg	5
9	wait-simple	5
10	wait-twice	5
11	multi-recurse	15
12	exit	5
13	halt	3

Functionality		
No.	Name	Point
14	create-empty	3
15	create-long	3
16	create-normal	3
17	create-exists	3
18	open-missing	3
19	open-normal	3
20	open-twice	3
21	read-normal	3
22	read-zero	3
23	write-normal	3

Functionality		
No.	Name	Point
24	write-zero	3
25	close-normal	3
26	rox-simple	3
27	rox-child	3
28	rox-multichild	3
Total Score		108

Evaluation

Robustness		
No.	Name	Point
1	exec-bad-ptr	3
2	exec-missing	5
3	sc-bad-arg	3
4	sc-bad-sp	3
5	sc-boundary	5
6	sc-boundary-2	5
7	wait-bad-pid	5
8	wait-killed	5

Robustness		
No.	Name	Point
9	close-stdin	2
10	close-stdout	2
11	close-bad-fd	2
12	close-twice	2
13	read-bad-fd	2
14	read-stdout	2
15	write-bad-fd	2
16	write-stdin	2
17	multi-child-fd	2
18	create-bad-ptr	3

Robustness		
No.	Name	Point
19	open-empty	2
20	open-bad-ptr	3
21	read-bad-ptr	3
22	write-bad-ptr	3
23	create-bound	3
24	open-boundary	3
25	read-boundary	3
26	write-boundary	3
27	create-null	2
28	open-null	2

Robustness		
No.	Name	Point
29	bad-read	1
30	bad-read2	1
31	bad-write	1
32	bad-write2	1
33	bad-jump	1
34	bad-jump2	1
Total Score		88

Evaluation

User Program

No-vm		
No.	Name	Point
1	multi-oom	1
Total Score		1

Filesys

Base		
No.	Name	Point
1	sm-create	1
2	sm-full	2
3	sm-random	2
4	sm-seq-block	2
5	sm-seq-random	3
6	lg-create	1
7	lg-full	2
8	lg-random	2
9	lg-seq-block	2
10	lg-seq-random	3

Base		
No.	Name	Point
11	syn-read	4
12	syn-write	4
13	syn-remove	2
Total Score		30

# of Total Tests	76
------------------	----

Evaluation

- Total development score :

$$\left(\frac{\text{Functionality}}{108} \times 35 + \frac{\text{Robustness}}{88} \times 25 + \frac{\text{no-vm}}{1} \times 10 + \frac{\text{filesystem(base)}}{30} \times 30 \right) / 100 \times 80$$

- Documentation
 - Use the document file uploaded on e-class.
 - Documentation accounts for 20% of total score.
(Development 80%, Documentation 20%)

Documentation

- Use the document file uploaded on e-class
- Documentation accounts for 20% of total score.
(Development 80%, Documentation 20%)

Submission

- We provide the script 'submit.sh' to make tar.gz file which contains 'src' directory and document file.

학생들의 편의를 위해 pintos 디렉토리 내 submit.sh 스크립트를 제공합니다.

이 스크립트는 src 디렉토리와 document file을 포함한 tar.gz 파일을 생성합니다.

Submission

- It is a **individual project**.
- Due date : **2026. 10. 4. 23:59**
- Submission
 - The form of submission file is as follows:

Name of compressed file	Example (ID: 20189999)
os_prj1_[ID].tar.gz	os_prj1_20189999.tar.gz

- No hardcopy.
- **Copy will get a penalty (1st time: 0 Point and downgrading, 2nd time: F grade)**

Submission

- **Contents**

- ① Pintos source codes (Only '**src**' **directory** in pintos directory)
최소한의 용량을 위해 **src 디렉토리만** 압축파일에 포함합니다.
- ② Document: **[ID].docx** (e.g. 20189999.docx; Other format is not allowed such as .hwp)

- **How to submit**

- 1) Make tar.gz file.

- Copy the document file ([ID].docx) to pintos directory.
- **Execute submit.sh script in the pintos directory and follow the instructions of the script.**
pintos 디렉토리 내의 submit.sh 스크립트를 실행하고 스크립트의 지시를 따르십시오.
- Check that **os_prj1_[ID].tar.gz** is created.
- Decompress **os_prj1_[ID].tar.gz** and check the contents in it. (\$ **tar -zxf os_prj1_[ID].tar.gz**)
(Only **[ID].docx** and **src** directory should be contained in the tar.gz file.)
- For example, if your ID is 20189999, os_prj1_20189999.tar.gz should be created.
To decompress the tar.gz file, execute **tar -zxf os_prj1_20189999.tar.gz**
- **Please check the contents of tar.gz file after creating it.**

- 2) Upload the **os_prj1_[ID].tar.gz** file to e-class.

5% of point will be deducted for a wrong form and way to submit.

❖ Late submission is allowed up to 3 days (~10/7, 23:59) and 10% of point will be deducted per day.

Submission

- **Notice – 'submit.sh'**

- The 'submit.sh' script should be executed on a directory where 'src' folder is located.
submit.sh 스크립트는 src 폴더가 위치한 디렉토리에서 실행되어야 합니다.

- **'ID' folder should not be in the directory. ('ID' folder will be removed after compressing process.)**

해당 디렉토리에 '학번' 폴더가 없어야 합니다. (압축 과정 중 '학번' 폴더를 생성하여 필요한 파일을 넣고 압축한 뒤 '학번' 폴더를 삭제합니다.)

- 'ID.docx' file should be located in the directory.

Also, report file with extensions other than 'docx' will not be compressed.

해당 디렉토리에 '학번.docx' 파일이 있어야 함께 압축됩니다.

또한 'docx' 이외의 확장자를 가진 보고서 파일은 압축되지 않습니다.

- Be sure to backup your code in case of an unexpected situation.
만일의 경우를 대비해 반드시 코드를 백업하여 주세요.

Submission

- **Disclaimer**

- You must check the contents of the tar.gz file before submission.
- 제출하기 전, tar.gz 파일의 내용물을 반드시 다시 한 번 체크하기 바랍니다.
- Any result produced from the 'submit.sh' script is at your own risk.
- 'submit.sh' 스크립트로 생성된 결과의 모든 책임은 사용자에게 귀속됩니다.